SLIC superpixel with C++ OpenMP

Parallel programming

Albi Sadikaj

UniFi – Aprile 2026

Introduction

What is SLIC?

SLIC is an image segmentation algorithm that groups pixels into **superpixels**; compact regions that respect object boundaries.

How does it work?

The way it works is by adapting k-means clustering in a 5-dimensional space combining color (CIE-Lab: L, a, b) and spatial position (x, y).
Each pixel is assigned to the nearest cluster within a local 2S × 2S search window, making the algorithm scale linearly with the number of pixels.

For what is it useful?

SLIC allows to reduce the complexity of subsequent computer vision tasks like segmentation and object detection.

Formulas

SLIC requires a total of 4 formulas:

Grid intervals

Since SLIC takes as input the number of clusters desired, we can calculate the step between each center with:

$$S = \sqrt{\frac{N}{K}}$$

Distance measure

To know in which cluster a pixel is, we need to measure the distance between it and the center of the cluster. To do so we use the following formula:

$$d_{\mathrm{lab}} = \sqrt{(l_{\mathrm{k}} - l_{\mathrm{i}})^2 + (a_{\mathrm{k}} - a_{\mathrm{i}})^2 + (b_{\mathrm{k}} - b_{\mathrm{i}})^2}$$

$$d_{xy} = \sqrt{(x_k - x_i)^2 + (y_k - y_i)^2}$$

$$D_s = d_{lab} + \frac{m}{S} d_{xy}$$

The m is a variable that can be set with a value from 1 to 20. Higher values make the distance depend more on the color.

Formulas

SLIC requires a total of 4 formulas:

Gradient

To find the best match in the 3×3 neighborhood, we use the gradient of each pixel and use the smallest value:

$$\mathrm{G(x,y)} = \left|\left|\mathrm{I(x{+}1,y)} - \mathrm{I(x{-}1,y)}\right|\right|^2 + \left|\left|\mathrm{I(x,y{+}1)} - \mathrm{I(x,y{-}1)}\right|\right|^2$$

Where I(x, y) is the lab vector at (x,y) and $||.||$ is the L_2 norm.

Error calculation

At every iteration we calculate the error using the L_1 distance (or Manhattan distance):

$$d_{L_1}(p,q) = \left|\left|p - q\right|\right|_{L_1} = \sum_{i=1}^{n} |p_i - q_i|$$

Algorithm

The algorithm behind SLIC is quite simple and is the following:

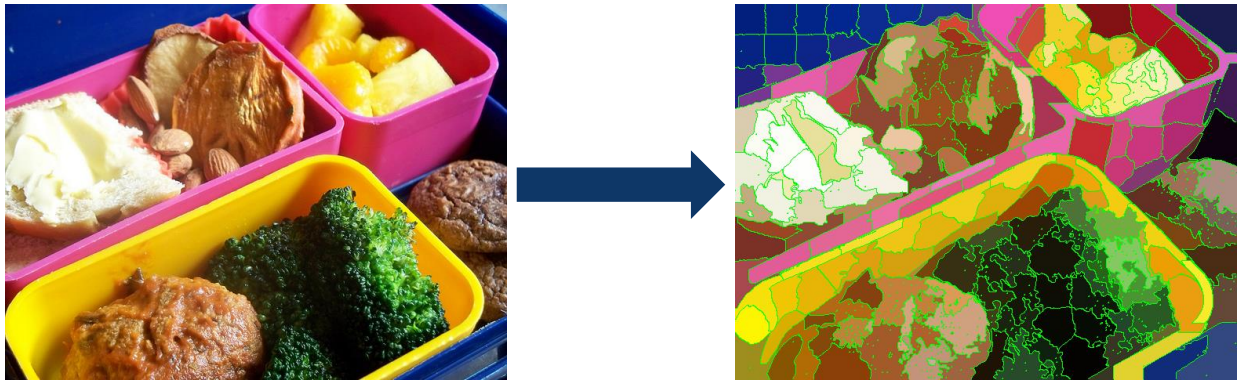
1: Initialize cluster centers $C_k = [l_k, a_k, b_k, x_k, y_k]^T$Ck = [lk, ak, bk, xk, yk]T by sampling pixels at
regular grid steps S.
2: Perturb cluster centers in an n × n neighborhood, to the lowest gradient position.
3: **repeat**
4: **for** each cluster center C_k **do**
5: Assign the best matching pixels from a 2S × 2S square neighborhood around the cluster center according to the distance measure.
6: **end for**
7: Compute new cluster centers and residual error E {L_1 distance between previous centers and
recomputed centers}
8: **until** $E \leq threshold$
9: Enforce connectivity.

A total of three implementation have been done: sequential, standard parallel and tile-based parallel

Sequential

Approach

The sequential is a 1:1 implementation of the algorithm defined earlier. The only change made is on the distance measure formula where, instead of doing all the square roots, the values were all squared, reducing complexity.



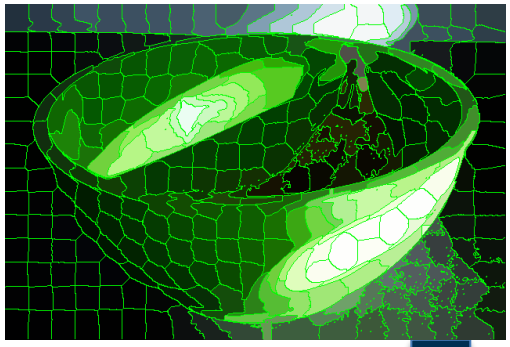
Example of SLIC sequential applied to COCO-1 with K=200

Standard parallel

Approach

This version of the parallel is a simple parallelized version of the sequential with no major changes using OpenMP.

The main steps to parallelize were:
- Step 5: best matching pixels
- Step 6: update cluster center and error
- Step 9: connectivity



Step 5

This step is important because it's the computationally intensive.
Luckily it's also easy to parallelize it, needing only:
- #pragma omp parallel for schedule(dynamic)
- double-checked locking with std::mutex

Step 6

This step has two for loops, both handled in the same way:
- #pragma omp for loop schedule(dynamic)
- reduction to handle the creation and aggregation of all support vector

Standard parallel

Step 9

The connectivity is the hardest step to parallelize and can't be done completely. The steps are the following:

Save in a vector all islands of pixels of every cluster → Remove the biggest island from the vector → Reapply distance measure to the remaining islands to assign them to new clusters

The problem in parallelizing is mostly the first part, since it requires the use of the Breadth-First Search (BFS) algorithm to get fill the vector, which it's not parallelizable.

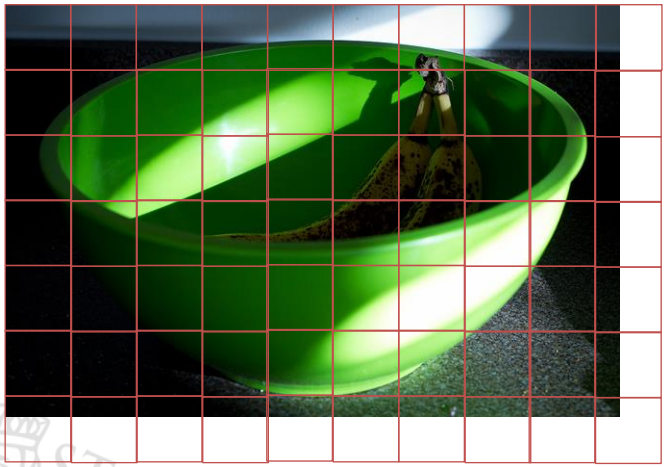
Solution? Instead of trying to parallelize the BFS itself, parallelize che calling of the BFS.
Since the islands of each cluster are completely separate from each other, we can assign a thread to each vector of island of each SP. This allows us to avoid having to run this part with a single-thread and, instead, just call the BFS with multiple threads.

Tile-based parallel

Approach

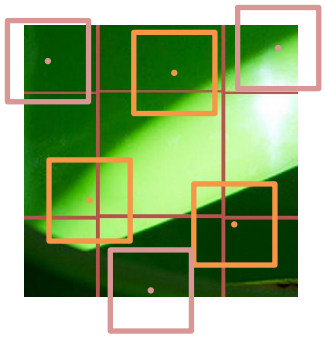
As we said, in the standard parallel we needed to use a mutex to avoid race conditions during step 5.

Generally this should be avoided since mutexes or reduction are not cheap and in this case, one way to avoid any race condition is with a tile-based approach. If we divide an image in small tiles K×K area, distanced from each other by 2K, we can assign that tile to a thread. This method removes completely any race conditions.



Once all tiles have been created, each one checks for which clusters centers area it touches.
If a cluster center area is touched, then it's added to the list of SP candidates.

Once all tiles are set up, each thread goes goes through the pixel in its tiles and assigns it to the best candidate from the small list.



In this case, the centers in light red will not be considered by the tile, while the ones in orange will.

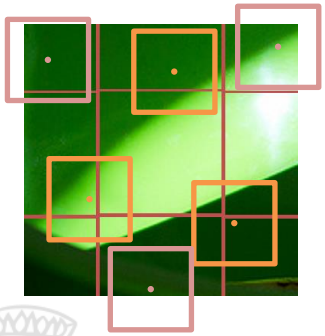
Tile-based parallel

Problems

While the tile-based seems like a perfect solution, it has a couple of problems that make it slower than the mutex version:

1) It requires a bigger overhead since each thread get its own vector to handle the candidate
2) The distance of the pixels is calculated with every candidate, no matter how likely it is.

Going back to the previous zoom:



Here we have 3 candidates for the pixels in tile.
Each pixel will calculate the distance from each of them and mantain the smallest, even if the pixel is top right and we're checking for the cluster in the bottom right.

Memory layout: SoA vs AoS

Key architectural decision

AoS (Array of Structures) — traditional layout:

```
struct pixel { float L, a, b; int x, y; };
```

SoA (Structure of Arrays) — adopted layout:

```
float L[], a[], b[];   int x[], y[];
```

SoA for the image buffer: no significant benefit.

- Pixels accessed sequentially → hardware prefetcher hides latency even with AoS

SoA for cluster centroids: small gains.

- Primitive contiguous arrays unlock native OpenMP array reductions
  `reduction(+: ptr_l[:k], ptr_a[:k], ptr_b[:k], ...)`
- Each thread accumulates independently; OpenMP merges at the end → fully lock-free

Benchmark setup

Parameters Tested

Thread Count: 1 to 16 threads (OMP only)
Images: COCO-1, 640x480, 200 SP; COCO-2, 640x429, 300 SP;
1080p, 1920x1080, 1000 SP; 4K, 3840x1609, 2000 SP;

Hardware Setup

CPU: AMD Ryzen 7 7840HS (8 cores)
Method:

- OpenMP: 15 averaged runs + 2 warmups

Sequential baseline results

Performance on 4 image resolutions

Image	Time ms
COCO-1	49.73
COCO-2	41.49
1080p	325.41
4k	900.04

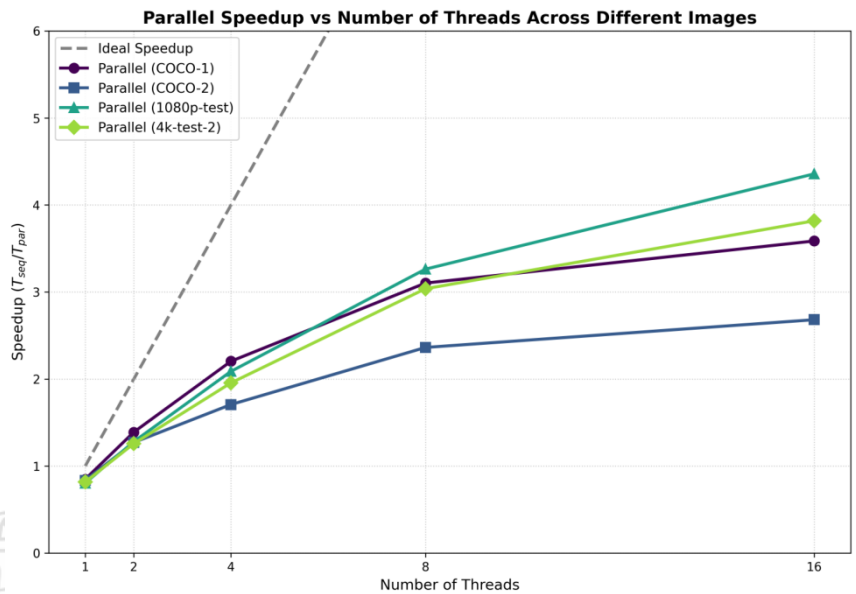
A key observation here is that times scale far more with the size of the image than the number of SP.

This is due to the fact that adding superpixels only adds so many operations, while increases in size add checks for each superpixles.

Standard parallel results

Image	T=1	T=2	T=4	T=8	T=16
COCO-1	58.57	35.77	22.56	16.03	13.87
COCO-2	49.70	32.64	24.30	17.56	15.47
1080p	404.59	254	155.81	99.74	74.68
4k	1101.7	715.44	460.12	296.30	235.73

Times in ms



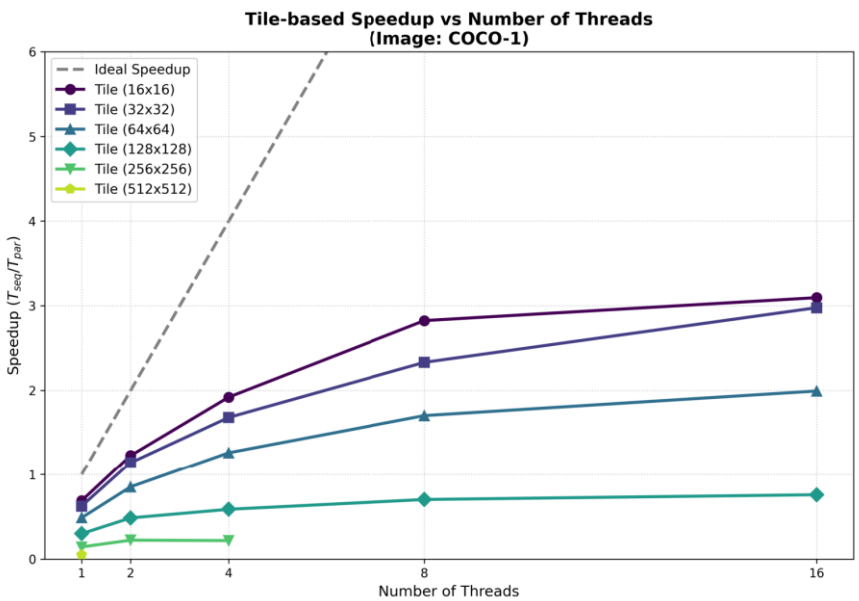
Like expected 1 thread, runs are slower than sequential due to OpenMP and mutex overheads.

Speedup drops after 8 threads. This is due to SMT. The two threads in a core also share L1/L2 cache and memory bandwidth. This is detrimental to SLIC since it's memory-bound.

Tile-based parallel results

Image	Tile size 16	32	64	128	256	512
COCO-1	72.13	79.50	101.95	165.58	345.07	933.98
COCO-2	89.86	114.27	205.09	499.41	1328.09	4175.17
1080p	573.78	637.82	690.73	1122.53	2420.78	6583.83
4k	1851.1	1738.38	1789.14	2736.99	5610.86	15013.7

Times tested with 1 thread in ms



Just like the standard parallel version, with 1 thread times ar higher than the sequential version.
In the case of the tile-based this problem is even more pronounced

The best tile size is generally 16, with 32 being marginally better for higher resolution images.
As we can see, tile is slower due to the needed overheads

Image	Tile size 16	32	64	128	256	512
COCO-1	16.06	16.70	24.95	65.49	-	-
COCO-2	19.27	24.75	58.21	226.54	-	-
1080p	85.67	89.08	109.15	270.65	907.247	-
4k	305.26	301.01	335.16	506.13	1491.62	7095.79

Times tested with 16 thread in ms. '-' are skipped times due to tiles too big.

Conclusions

Summary and lessons learned

What was achieved:
- Full parallel SLIC with OpenMP: sequential, standard parallel, tile-based
- 3.82x peak speedup on 4K images at 16 threads
- Lock-free tile variant designed, benchmarked and fully analyzed

Best configuration:
- Standard parallel with row-level mutexes and SoA centroid layout

Key lessons:
- Lock-free ≠ faster: geometric overhead outweighed synchronization savings
- SoA matters for centroids, not for the image, due to prefetcher hiding latency